JOINT AGENT MEMORY AND EXPLORATION LEARNING VIA NOVELTY SIGNALS

Shizuo Tian[1], Xiaohong Weng[2], Rui Kong[3], Yuxuan Chen[1], Guohong Liu[1], Yuebing Song[4], Jiacheng Liu[5], Yuchen Li[3], Dawei Yin[3], Ting Cao[1], Yunxin Liu[1], Yuanchun Li[1]*
[1]Tsinghua University, [2]Sun Yat-sen University, [3]Baidu Inc., [4]Tongji University,
[5]Peking University

ABSTRACT

In open-ended environments, exploration is fundamental for autonomous agents, yet current language model agents struggle with this. Effective exploration requires memory, but retaining raw interaction histories is computationally expensive over long trajectories. While latent memory offers a solution to compress interaction histories, its training lacks reliable supervisory signals. We introduce **J**oint **A**gent **M**emory and **E**xploration **L**earning (**JAMEL**), a framework that trains agentic memory and exploration policy together through novelty-driven interaction. We observe that memory and exploration form a mutually dependent loop: sustained exploration requires memory to distinguish exhausted behaviors from unseen ones, while novelty-seeking interaction provides the supervision needed to make memory useful for future exploration. By utilizing deterministic and persistent novelty signals such as code coverage in the GUI domain, we provide natural, annotation-free supervision for the memory module. Empirical evaluations demonstrate that JAMEL successfully generalizes to unseen environments. Its exploration capability outperforms open-weight baselines and rivals the exploration depth of a closed-source model while reducing token consumption. Our code and model are open-sourced at `https://github.com/MobileLLM/JAMEL`.

1 INTRODUCTION

Exploration is a fundamental capability for intelligent agents operating in open-ended environments, where extrinsic rewards are extremely sparse or absent (Pathak et al., 2017). Recent work has shown that current LLM-based agents lack this capability: even when agents encounter unexpected but task-relevant information during interaction, they fail to act on it in the majority of cases (Engländer et al., 2026), a deficit rooted in how agents are trained rather than in inference-time configuration.

Effective exploration requires memory. In a partially observable environment, an agent cannot decide whether an action is worth trying unless it knows which states, interface regions, or behavioral consequences have already been observed. The most direct solution is to retain the full interaction history, but this becomes computationally expensive as trajectories grow longer. Agent memory is therefore needed to compress long histories into a more compact form. Agent memory has received growing attention (Zhang et al., 2024), and latent memory in particular has been studied for its efficiency in compressing interaction history into a vector prefix (Zhang et al., 2026). However, the difficulty is that agent memory lacks reliable step-level supervision: we usually do not know what each memory state should encode, or how the policy should use them in future decisions. This problem becomes more severe over long trajectories, where ineffective memory causes the agent to revisit exhausted behaviors and make poor decisions. Explicit textual memories are interpretable: their contents can be inspected, revised, or heuristically filtered when the agent fails. Latent memory is usually more efficient but much harder to supervise, because its compressed vectors lack human-readable semantics. As a result, standard task demonstrations provide no clear step-level target for what the memory should encode or how the policy should use it to avoid repetition and discover new states.

*Corresponding Author: Yuanchun Li (liyuanchun@air.tsinghua.edu.cn)

We observe that exploration and memory are mutually dependent. Memory enables the agent to avoid repetition and discover unexplored behaviors, while exploration exposes which historical information is useful for future decisions. Based on this observation, exploration itself can provide supervision for memory. Novelty is awarded only when the agent reaches behavior not covered by its own history, maximizing this reward forces the memory-conditioned policy to encode and use what has already been tried. This alignment also creates a natural curriculum: as familiar interactions are exhausted, only deeper multi-step sequences continue to yield novelty reward, driving the policy and memory to improve together without explicit curriculum design. In some environments, a suitable novelty signal is available without annotation effort. In the GUI domain, code coverage provides a natural proxy: any software application can be instrumented to report which code paths have been executed, yielding a deterministic and persistent measure of behavioral novelty. In embodied environments, analogous signals arise from discovering new states or objects encountered during navigation or manipulation.

We introduce **J**oint **A**gent **M**emory and **E**xploration **L**earning (**JAMEL**) to instantiate this idea, and our contributions are as follows: (1) We design a latent memory architecture that compresses historical information into memory tokens, substantially reducing the computational overhead of processing long interaction histories. (2) We build a data collection pipeline for training JAMEL's exploration ability via rejection fine-tuning, collecting 24k training samples across 86 web applications in the GUI domain. (3) We show that JAMEL generalizes exploration to unseen applications, outperforming existing agent memory baselines on 10 held-out apps.

2 Related Work

Agent Memory Efficiently compressing and organizing long-term interaction histories is fundamental to enabling autonomous agents to learn continuously. Early approaches rely on fixed context windows (Beltagy et al., 2020) or external RAG retrieval (Asai et al., 2023), which scale poorly with interaction length. In contrast, prompt tuning methods (Liu et al., 2022; 2023) introduce trainable soft prefixes for parameter-efficient adaptation. To model longer dependencies, recurrent memory transformers (Bulatov et al., 2022) propagate summary states across segments, while recent advances integrate explicit or procedural memory tokens directly into transformer layers. For instance, MemoryLLM and M+ (Wang et al., 2024; 2025) maintain layer-wise latent memory pools with controlled forgetting, and TokMem (Wu et al., 2026) tokenizes procedural skills for continual adaptation. Token Memory (Sun et al., 2025a) further stores contextual information to guide generation. However, these mechanisms typically decouple memory management from the agent's learning objective, relying on static buffers or heuristic retrieval rules. Our approach integrates memory compression directly into the exploration loop, enabling the agent to autonomously consolidate high-value trajectories without external annotation.

Exploration Policy Efficient exploration remains a core challenge in sparse-reward environments where agents must discover valuable states without explicit supervision. Classical approaches propose intrinsic motivation signals like ICM or RND (Pathak et al., 2017; Burda et al., 2018) to encourage novelty through prediction or random network errors. To accelerate search under sparse rewards, archive-based methods like Go-Explore (Ecoffet et al., 2020) maintain frontier states for targeted exploration, while variants such as IGE (Lu et al., 2025) incorporate LLM-based similarity judgments, XTX (Tuyls et al., 2022) combines imitation learning with curiosity, and GLoW (Kim & Hwang, 2025) leverages dual-scale world models for maintaining high-value discoveries and learning from local trial-and-error. Tree-search alternatives like MC-LAVE and MC-DML (Jang et al., 2021; Shi et al., 2025) further remove the dependency on state rollback. In the GUI domain, GUI-Xplore constructs exploration videos and hierarchical downstream tasks to improve GUI agents' cross-application and cross-task generalization (Sun et al., 2025b), while LLM-Explorer shows that compact knowledge maintained by LLMs can guide efficient app exploration with much lower cost than step-by-step LLM action generation (Zhao et al., 2025). Our work follows this direction but focuses on a different question: how can novelty signals supervise latent agent memory so that exploration-derived knowledge can be encoded into compact memory tokens and used by the policy for future exploration?

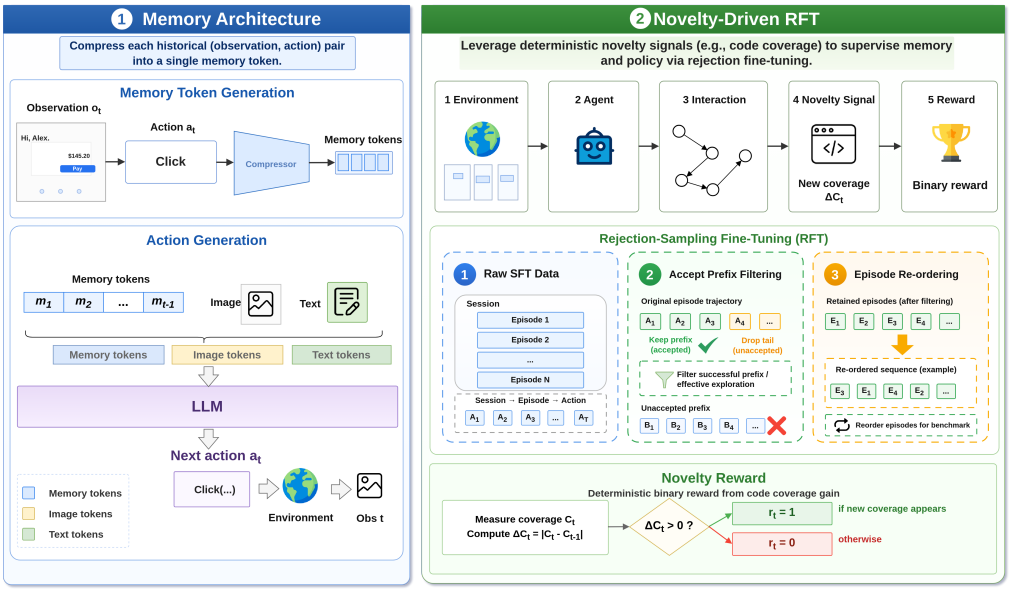

Figure 1: **Architecture of JAMEL.**

3 METHODOLOGY

3.1 EXPLORATION PROBLEM

We model the exploration problem as a finite-horizon partially observable Markov decision process, $\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, \Omega, \rho_0, H)$. At step t, the environment has hidden state $s_t \in \mathcal{S}$, emits an observation $o_t \sim \Omega(\cdot \mid s_t)$, receives an action $a_t \in \mathcal{A}$, and transitions according to $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. The agent observes the current observation and the previous interaction history

$$\mathcal{H}_{<t} = ((o_1, a_1), \ldots, (o_{t-1}, a_{t-1})), \tag{1}$$

and samples actions from $\pi(\cdot \mid o_t, \mathcal{H}_{<t})$. Exploration has no task-specific goal state. Instead, the objective is to expose behavior that has not appeared earlier in the same session. We formalize this with a novelty score

$$r_t = \text{Novelty}(s_t, \mathcal{H}_{<t}) \in \mathbb{R}_{\geq 0}, \tag{2}$$

where larger values indicate that the current state is less familiar given the visited history. The novelty function is abstract and can be instantiated in different ways. ICM (Pathak et al., 2017), for example, uses world-model prediction error, while JAMEL uses rule-based signals derived from code coverage. The exploration objective is

$$J(\pi) = \mathbb{E}_{\tau \sim (\pi, \mathcal{P})} \left[\sum_{t=1}^{H} r_t \right]. \tag{3}$$

Under partial observability, the agent must infer from history which behavior units have already been exhausted.

3.2 MODEL ARCHITECTURE OF JAMEL

Let the agent's interaction history up to step t be $H_{<t} = \{(o_1, a_1), \ldots, (o_{t-1}, a_{t-1})\}$. For each historical step, we first serialize the observation-action pair into an input sequence

$$x_i = \text{Format}(o_i, a_i).$$

A frozen vision-language model F_ϕ is then used as the history compressor. We feed x_i into F_ϕ and take the final-layer hidden state of the end-of-sequence token as a compact representation of this step:

$$h_i = F_\phi(x_i)_{\text{EOS}} \in \mathbb{R}^{d_c}.$$

This EOS representation serves as a single latent memory token for the corresponding historical interaction. In this way, each potentially long observation-action pair is compressed into one vector, while the compressor itself remains fixed.

The memory state at time t is the sequence of all the memory tokens:

$$\mathbf{M}_t = [\mathbf{h}_1, \ldots, \mathbf{h}_{t-1}] \in \mathbb{R}^{(t-1)\times d_c}. \tag{4}$$

A learned linear aligner $\mathbf{W} \in \mathbb{R}^{d_c \times d_{\text{LM}}}$ projects the memory tokens into the policy's embedding space. The projected memory is prepended to the input embedding sequence,

$$\mathbf{e}_t = [\mathbf{M}_t\mathbf{W} \mid \text{Embed}(o_t)], \tag{5}$$

and the action is then sampled as $a_t \sim \pi(\cdot \mid \mathbf{e}_t)$.

3.3 Novelty-Based Intrinsic Reward

Given the agent's history $\mathcal{H}_{<t}$, we define the intrinsic novelty reward at step t as a binary signal indicating whether the action produced genuinely new experience:

$$r_t = \mathbf{1}\big[\text{Novelty}(o_t, a_t, \mathcal{H}_{<t}) > 0\big], \tag{6}$$

where Novelty measures how much unexplored behaviors the current step discovers relative to the accumulated history. The novelty measure should be persistent: once a state has been visited, it must never register as novel again, otherwise the agent can cycle through a small set of states and accumulate reward without genuine exploration.

In the GUI domain, code coverage provides a natural and deterministic proxy for novelty. Any software application can be instrumented to report which code paths have been executed; a step is novel if and only if it triggers at least one previously unexecuted path. Concretely, we define the cumulative coverage score at step t as

$$\mathcal{C}(t) = \text{cov}_{\text{lines}}(t) + \text{cov}_{\text{branches}}(t) + \text{cov}_{\text{statements}}(t) + \text{cov}_{\text{functions}}(t), \tag{7}$$

where each term counts the total number of coverage entities executed at least once across all steps up to and including t. The intrinsic reward then simplifies to

$$r_t = \mathbf{1}\big[\mathcal{C}(t) > \mathcal{C}(t-1)\big]. \tag{8}$$

We collect coverage data via the V8 JavaScript engine's coverage reports and compute $\mathcal{C}(t)$ using the Istanbul reporter (Istanbul Contributors, 2026). Steps with invalid actions or execution errors receive $r_t = 0$. The coverage baseline is maintained during exploration and is not reset when the browser returns to the application's start page, so already-explored code paths yield no reward in subsequent episodes.

Training JAMEL requires a dataset of exploration trajectories labeled with the intrinsic reward defined above. We collect this data by deploying a general-purpose LLM to explore each target application in a browser environment. The LLM is prompted to produce a chain-of-thought reasoning trace followed by a single browser action at each step, operating with the full history in its context window. After each step, the coverage reward r_t is computed according to equation 8.

A session consists of multiple episodes, each starting from the application's initial page and running for up to N steps before a browser reset. Because the coverage baseline is shared across episodes, the reward signal becomes progressively sparser as the session advances, forming a natural curriculum.

From each episode, we construct a training prefix by selecting steps 1 through t^*, where t^* is the index of the last step with $r_t > 0$. Episodes with no positive-reward step are discarded. This prefix selection ensures that every retained step belongs to a trajectory that eventually produces novelty reward, providing a coherent learning signal for both the policy and the memory.

For each retained step $t \leq t^*$, we pre-compute the memory tokens $\mathbf{M}_t$ by running the compressor F_ϕ on all steps that precede t in the session. The training sample for step t is the triple $(o_t, \mathbf{M}_t, a_t)$, and the training objective is to maximize the likelihood of the action a_t given the current observation and memory. The memory aligner $\mathbf{W}$ is updated jointly during this supervised phase.

We collect 24k training samples across 86 web applications on ScaleWoB (Liu, 2026) using this pipeline.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Benchmark. We evaluate on **ScaleWoB** (Liu, 2026), a benchmark of real web applications for evaluating computer-use agents. ScaleWoB includes 96 apps spanning e-commerce, social media and video (Weibo, Douyin, Zhihu, Youku, Tencent Video), travel and logistics (Amap, Cainiao, Expedia), productivity (Feishu/Lark, DingTalk, WPS), and a range of common apps. We partition into 86 training apps and 10 test apps. The evaluation of each app consists of $T = 50$ steps. Agents interact with the browser via the BrowserGym action space (de Chezelles et al., 2025; Drouin et al., 2024), which covers clicks, form fills, scrolls, navigation, etc. At each step, the agent receives the page accessibility tree (a11y tree) and the list of currently interactive element identifiers as its observation; the image-based variants additionally receive a screenshot.

Metrics. We report **cumulative coverage reward**: the total number of steps in a session at which the agent's action caused the cumulative JavaScript coverage score to increase. Formally, each step contributes $r_t = 1$ if $\mathcal{C}(t) > \mathcal{C}(t-1)$ and $r_t = 0$ otherwise. A higher value indicates that the agent triggers more distinct code paths across the session, reflecting stronger exploration ability.

Baselines. Baselines are evaluated under the same ScaleWoB environment and 50-step budget. As shown in Table 1, these include:

- **ReAct-text** and **ReAct-vision** (Yao et al., 2023): We implement the ReAct framework on top of the Gemini 3.1 Flash-Lite (Google DeepMind, 2026). The text variant provides the agent with the full session trajectory as an AXTree-only text prompt, retaining all prior (observation, think, action, reward) tuples within a token budget of $\sim$1M tokens and without dropping any step records. The ReAct-vision variant appends a webpage screenshot of the current observation.
- **MAI-UI** (Zhou et al., 2025): A family of foundation GUI agents (2B, 8B, 32B, and 235B-A22B variants) built on Qwen3-VL. MAI-UI employs a device/cloud collaboration mechanism that routes each step to either an on-device or cloud model based on task complexity. The complete interaction trajectory is stored locally and reformatted per model tier; on execution errors, an error summary is appended to the history before the next decision. We evaluate the 8B variant (MAI-UI-8B).
- **Mobile-Agent-v3.5** (Xu et al., 2026): An end-to-end GUI automation framework built on GUI-Owl-1.5 (available in 2B, 4B, 8B, 32B, and 235B variants). Mobile-Agent-v3.5 uses a *hierarchical context compression* strategy: the most recent steps retain full screenshots and action history, while earlier steps are distilled into concise action-conclusion summaries; a dedicated Notetaker module maintains a running log of task-critical information across steps. We evaluate the 8B variant (GUI-Owl-1.5-8B).

Implementation Details. We use Qwen3-VL-2B-Instruct (Bai et al., 2025) with $d_c = 2048$ as the history compressor. Each (observation, action) pair at step t is compressed into a single memory token. The memory tokens are projected into the embedding space of policy model, which is based on Qwen2.5-VL-7B-Instruct with $d_{\mathrm{LM}} = 3584$ via a learned linear aligner and prepended as a soft prefix. JAMEL-9B is then fine-tuned using 24k samples collected from 86 apps. During evaluation, it is tested on 10 unseen apps to assess its generalization capability.

4.2 MAIN RESULTS

As shown in Table 1, among small GUI agents, JAMEL achieves the highest reward, compared with MAI-UI and Mobile-Agent-v3.5. Against Gemini 3.1 Flash-Lite ReAct baselines, JAMEL remains competitive: it exceeds ReAct-text and trails ReAct-vision by only 0.2 reward, despite using a 2B memory compressor and a 7B decoder.

As illustrated in Figure 2, local baselines exhibit an early plateau due to premature exploration stagnation. This stagnation occurs plausibly because these methods prune their context, inevitably discarding critical historical information as the session progresses. Cloud baselines avoid this issue

Table 1: **Main results.** Cumulative coverage reward averaged over 10 test apps over 50 steps. **Bold** marks the best result and underline marks the second-best result.

Method	Model	Memory	Avg. reward
Closed-source Models			
ReAct-text	Gemini 3.1 Flash-Lite	Full history	19.9
ReAct-vision	Gemini 3.1 Flash-Lite	Full history	**20.9**
Open-Source Models			
MAI-UI	MAI-UI-8B	Device/cloud routing	8.4
Mobile-Agent-v3.5	GUI-Owl-1.5-8B	Sliding window + Notetaker	5.9
JAMEL	JAMEL-9B	Latent memory	20.7

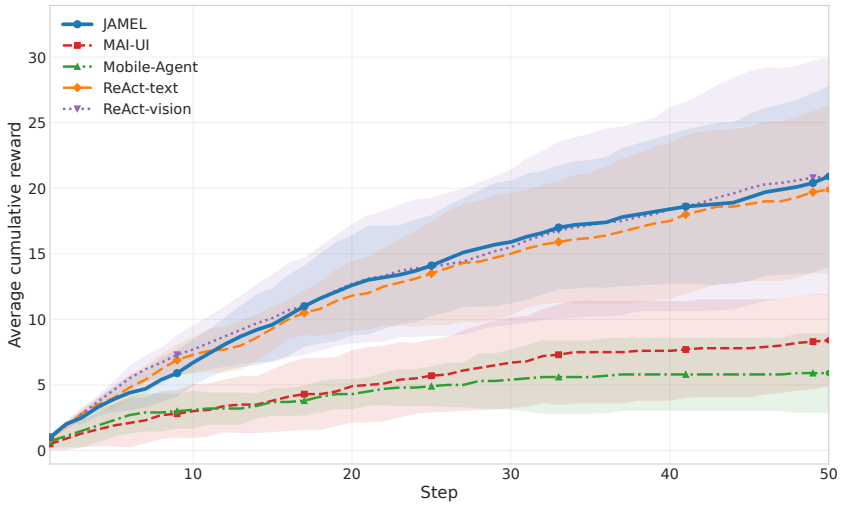

Figure 2: **Reward accumulation on test apps.** Average cumulative coverage reward across 10 test apps over a 50-step session. Shaded bands denote standard error across apps.

by retaining the complete explicit interaction history without any pruning. JAMEL aligns with this comprehensive retention strategy by avoiding context truncation. Instead, JAMEL compresses all historical information into latent memory tokens. This mechanism allows JAMEL to continuously uncover new application states without stalling, maintaining a steady and continuous upward trajectory that closely tracks the performance of the cloud models, successfully achieving an exploration depth highly competitive with cloud baselines while remaining significantly more efficient.

4.3 Analysis

Reward curve and the natural curriculum. Figure 2 shows average cumulative reward curve for our method and baselines on the test apps. Reward accumulates rapidly in early steps as shallow, easily-triggered interactions are exhausted, then slows as the agent must discover deeper multi-step paths. This sparsifying dynamic acts as a natural curriculum: the training signal becomes progressively harder, compelling the policy to develop richer exploration strategies over time.

Token efficiency. Table 2 reports the total input tokens consumed over all 500 evaluation steps and the corresponding per-step average. ReAct-text and ReAct-vision incur the largest context cost because they retain long explicit histories and screenshots. Mobile-Agent-v3.5 and MAI-UI reduce this cost, but still require 2.76× and 2.81× more tokens than JAMEL, respectively. Our latent prefix keeps context compact, making exploration substantially more token efficient.

Table 2: **Token Consumption.** Input token consumption over 10 test apps with 50 steps per app. Lower is better. **Bold** marks the best result.

Method	Input Tokens	Avg. per Step	Rel. to JAMEL
Mobile-Agent-v3.5	2,931,946	5,863.9	2.76×
MAI-UI	2,980,061	5,960.1	2.81×
ReAct-Text	18,938,833	37,877.7	17.85×
ReAct-Vision	23,260,296	46,520.6	21.92×
JAMEL	**1,061,103**	**2,122.2**	**1.00×**

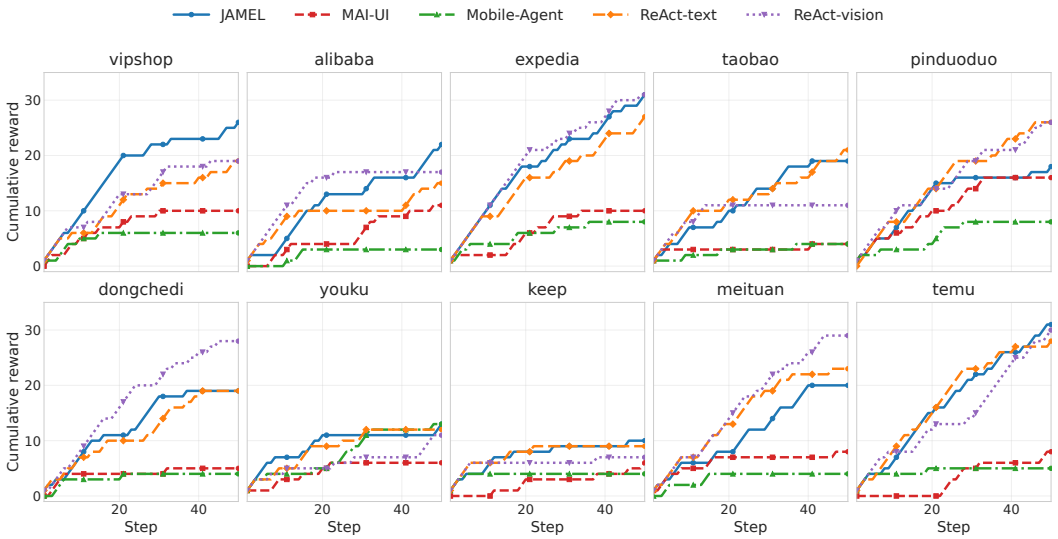

Figure 3: **Per-app reward accumulation.** Cumulative coverage reward trajectories evaluated on individual unseen applications.

Exploration Patterns. The per-app breakdown in Figure 3 reveals distinct structural exploration patterns dictated by varying software environments. In structurally deep commerce and travel platforms such as Vipshop, Expedia, and Temu, JAMEL maintains continuous upward trajectories, demonstrating its capacity for sustained sequential navigation. Convers ely, media and lifestyle applications like Youku and Keep impose a natural exploration ceiling where all methods inevitably plateau early probably due to inherently limited interactive depth. Furthermore, the stepwise reward surges observed in Alibaba and Taobao highlight the ability of JAMEL to escape local optima. While context pruning causes local baselines to stagnate within initial screens, the latent memory of JAMEL enables the policy to synthesize past interactions and transition into new application modules. However, JAMEL remains occasionally susceptible to entrapment. As observed in Pinduoduo, JAMEL experiences prolonged plateaus, suggesting that exceptionally dense interfaces can still challenge the compressed memory representation and temporarily hinder continuous state discovery.

Case Study. A detailed examination of the failure modes highlights specific interaction challenges within complex environments like Pinduoduo. The primary obstacle in such applications arises from persistent modal overlays. In these scenarios, the agent frequently attempts to interact with background elements that appear visually available but are rendered functionally unresponsive by the active foreground window. Conversely, applications featuring straightforward graphical layouts without frequent overlay interruptions, such as Expedia, facilitate highly efficient exploration. Within these cleaner environments, JAMEL systematically leverages persistent structural components like bottom navigation menus to transition seamlessly between distinct functional modules.

5 DISCUSSION

Scaling Laws of Exploration. A promising direction for future research lies in exploring the scaling laws of novelty-driven memory architectures like JAMEL. Integrating Reinforcement Learning (RL) strategies presents a natural evolution. Because the novelty reward inherently provides a natural curriculum, it facilitates the progressive learning of complex environment operations—advancing smoothly from shallow interactions to deep, multi-step navigation. Investigating how larger model capacities, scaled-up autonomous data collection and more exploration steps can further benefit from this novelty-guided curriculum remains an open frontier for future work.

Memory-Conditioned Task Execution and Continual Learning. Furthermore, the latent memory generated during the exploration process holds significant potential to benefit specific downstream tasks. This naturally points toward an "explore-then-execute" paradigm: a model first autonomously explores an unknown environment to accumulate structural memory, and subsequently relies on these exploration results to execute specific user instructions. Such an approach offers a pathway for agent self-evolution and continual learning. Developing algorithms that drive models to autonomously explore new environments and internalize new capabilities stands as a critical future direction for enabling rapid adaptation to long-tail scenarios and reducing the reliance on human-annotated trajectories.

6 CONCLUSION

We presented JAMEL, a framework for training agentic latent memory via novelty-driven exploration. We addressed the lack of explicit supervision for memory modules by demonstrating that persistent novelty signals, such as application code coverage in the GUI domain, can supervise agentic memory and exploration policy together by rewarding actions that use past experience to discover unexplored behaviors. Our empirical results show that JAMEL successfully generalizes to unseen environments, outperforming current open-weight baselines and rivaling the exploration depth of a closed-source model while reducing token consumption. While dense interfaces with persistent modal overlays can occasionally challenge the compressed representation, JAMEL establishes a scalable paradigm for autonomous agents in which memory and exploration are not separate modules, but mutually reinforcing capabilities learned through persistent novelty signals.

REFERENCES

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2023. URL https://arxiv.org/abs/2310.11511.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. URL https://arxiv.org/abs/2511.21631.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer, 2022. URL https://arxiv.org/abs/2207.06881.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation, 2018. URL https://arxiv.org/abs/1810.12894.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=5298fKGmv3. Expert Certification.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: how capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580 – 586, 2020. URL https://doi.org/10.1038/s41586-020-03157-9.

Leon Engländer, Sophia Althammer, Ahmet Üstün, Matthias Gallé, and Tom Sherborne. Agents explore but agents ignore: Llms lack environmental curiosity, 2026. URL https://arxiv.org/abs/2604.17609.

Google DeepMind. Gemini 3.1 Flash-Lite: Model Card. https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/, March 2026. Published March 2026. Accessed: 2026-05-30.

Istanbul Contributors. Istanbul: JavaScript test coverage made simple. https://istanbul.js.org/, 2026. Software project website. Accessed: 2026-05-30.

Youngsoo Jang, Seokin Seo, Jongmin Lee, and Kee-Eung Kim. Monte-carlo planning and learning with language action value estimates. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=7_G8JySGecm.

Minsoo Kim and Seung-won Hwang. Dual-scale world models for llm agents towards hard-exploration problems, 2025. URL https://arxiv.org/abs/2509.24116.

Guohong Liu. ScaleWoB: Scalable world-of-bit for evaluating computer-use agents, 2026. URL https://github.com/ScaleWoB/ScaleWoB. Python SDK and benchmark repository. Accessed: 2026-05-30.

Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks, 2022. URL https://arxiv.org/abs/2110.07602.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too, 2023. URL https://arxiv.org/abs/2103.10385.

Cong Lu, Shengran Hu, and Jeff Clune. Intelligent go-explore: Standing on the shoulders of giant foundation models, 2025. URL https://arxiv.org/abs/2405.15143.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML'17, pp. 2778–2787. JMLR.org, 2017.

Zijing Shi, Meng Fang, and Ling Chen. Monte carlo planning with large language model for text-based game agents, 2025. URL https://arxiv.org/abs/2504.16855.

Taize Sun, Katsuhide Fujita, Konstantin Markov, and Shengbo Chang. Token memory transformer with infinite context. In *Advanced Intelligent Computing Technology and Applications: 21st International Conference, ICIC 2025, Ningbo, China, July 26–29, 2025, Proceedings, Part XXIV*, pp. 319–330, Berlin, Heidelberg, 2025a. Springer-Verlag. ISBN 978-981-95-0019-2. doi: 10.1007/978-981-95-0020-8_27. URL https://doi.org/10.1007/978-981-95-0020-8_27.

Yuchen Sun, Shanhui Zhao, Tao Yu, Hao Wen, Samith Va, Mengwei Xu, Yuanchun Li, and Chongyang Zhang. Gui-xplore: Empowering generalizable gui agents with one exploration, 2025b. URL https://arxiv.org/abs/2503.17709.

Jens Tuyls, Shunyu Yao, Sham Kakade, and Karthik Narasimhan. Multi-stage episodic control for strategic exploration in text games, 2022. URL https://arxiv.org/abs/2201.01251.

Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. Memoryllm: Towards self-updatable large language models, 2024. URL https://arxiv.org/abs/2402.04624.

Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. M+: Extending memoryllm with scalable long-term memory, 2025. URL https://arxiv.org/abs/2502.00592.

Zijun Wu, Yongchang Hao, and Lili Mou. TokMem: One-token procedural memory for large language models. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=RWjEf9PdiJ.

Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, Zhiyuan Chen, Jitong Liao, Qi Zheng, Jiahui Zeng, Ze Xu, Shuai Bai, Junyang Lin, Jingren Zhou, and Ming Yan. Mobile-agent-v3.5: Multi-platform fundamental gui agents, 2026. URL https://arxiv.org/abs/2602.16855.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL https://arxiv.org/abs/2404.13501.

Zeyu Zhang, Rui Li, Xiaoyan Zhao, Yang Zhang, Wenjie Wang, Xu Chen, and Tat-Seng Chua. Nextmem: Towards latent factual memory for llm-based agents, 2026. URL https://arxiv.org/abs/2603.15634.

Shanhui Zhao, Hao Wen, Wenjie Du, Cheng Liang, Yunxin Liu, Xiaozhou Ye, Ye Ouyang, and Yuanchun Li. Llm-explorer: Towards efficient and affordable llm-based exploration for mobile apps, 2025. URL https://arxiv.org/abs/2505.10593.

Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. Mai-ui technical report: Real-world centric foundation gui agents, 2025. URL https://arxiv.org/abs/2512.22047.